

Platform for Continuous Evaluation of Romanian Literature

Team Members

Iacob Teodora, Martinas Ioana Maria, Mercas Ioana Elisabeta,
Sandu Theodor, Sacarescu Rebecca Maria

Abstract: This project proposes a web platform for continuous evaluation of Romanian literature editions, combining automatic data ingestion, user reviews, ranking transparency, and search capabilities. The current implementation already defines an end-to-end architecture with a BeautifulSoup-based crawler for collecting metadata, a FastAPI backend for business logic, PostgreSQL for persistent storage, Meilisearch for full-text lookup, and a React frontend for interaction. Users can browse catalog entries, submit reviews, inspect rankings, and export datasets in CSV or JSON format. The platform includes practical trust and governance features such as rate-limited review submission, moderation endpoints for approval or rejection, and an audit trail that records score updates through score events. The scoring model applies Bayesian shrinkage to avoid unstable rankings when the number of reviews is small, while confidence values communicate ranking reliability. From a software engineering perspective, the project emphasizes modularity, API-first integration, and reproducible deployment with Docker and migration tooling. The NLP module is treated as optional but high-value: sentiment classification fine-tuned on LaRoSeDa and theme extraction from reviews, both integrated only after quality validation.

Keywords: Romanian literature; web platform; FastAPI; BeautifulSoup; PostgreSQL; Meilisearch; ranking transparency; moderation; audit trail; NLP

1 Main Text

The following document describes the architecture, implementation, and deployment strategy for a comprehensive platform designed to evaluate Romanian literature editions. The system integrates multiple technologies to provide an end-to-end solution that serves both casual readers and literary analysts.

1.1 Structure

The platform consists of several interconnected components working together to deliver a seamless experience. The crawler subsystem continuously harvests book metadata from major Romanian online bookstores including Bookzone, Carturesti, and Libris. This data feeds into a normalized database schema that eliminates duplicates through intelligent ISBN matching and title comparison algorithms.

The backend API layer, built on FastAPI, exposes RESTful endpoints for all core operations including edition retrieval, review submission, ranking calculation, moderation workflows, and data export. Authentication and authorization mechanisms protect sensitive operations while rate limiting prevents abuse. The scoring engine implements a Bayesian approach that balances review counts with average ratings, ensuring that books with few reviews are not unfairly penalized or promoted.

The frontend interface, developed with React and TypeScript, provides intuitive access to the catalog through multiple views: browsing by various criteria, full-text search powered by Meilisearch, detailed edition pages with aggregated reviews, and administrative panels for content moderation. User interactions such as review submission and helpful voting create engagement loops that improve content quality over time.

The search infrastructure relies on Meilisearch, a specialized engine optimized for typo tolerance and relevance ranking. Every ingested edition is automatically indexed, enabling users to find books by title, author, publisher, or ISBN with minimal latency even as the catalog scales to thousands of entries.

Data integrity and auditability are ensured through comprehensive logging. Every score recalculation generates an event record capturing the algorithm parameters and resulting values, allowing operators to trace ranking changes back to their causes. Similarly, all moderation actions are logged with timestamps and responsible moderator identifiers.

1.2 Tables

All database tables follow a consistent naming convention and are managed through Alembic migrations to ensure schema versioning and rollback capability. The core tables include books (canonical work metadata), editions (specific publication variants), authors (with many-to-many relationships to books), reviews (user-submitted ratings and text), reviewers (user profiles with reputation scores), score events (audit trail for ranking changes), and refresh tokens (for secure authentication flows).

Table 1: Core Database Entities	
Entity	Description
Book	Canonical work with title and associated authors
Edition	Physical or digital publication of a book with ISBN, publisher, year
Author	Writer with biographical information and linked works
Review	User evaluation with rating (1–5), text, sentiment score, approval status
Reviewer	User profile with authentication credentials and activity metrics
Score Event	Audit record for each ranking calculation with parameters and results

The relational structure maintains referential integrity through foreign key constraints while allowing flexible queries across entities. For example, retrieving all reviews for editions of a particular book requires joining the books, editions, and reviews tables, which PostgreSQL handles efficiently with appropriate indexing strategies.

1.3 Construction of References

References to external sources are constructed following standard citation formats. The project documentation acknowledges the LaRoSeDa dataset for Romanian sentiment analysis training data, the Meilisearch project for search infrastructure, and the FastAPI

framework for backend development. All external libraries and their versions are tracked in requirements files to ensure reproducible builds.

When referencing specific implementation details, the document provides file paths relative to the project root (e.g., `backend/app/api/rankings.py` for the ranking endpoint implementation) to facilitate code review and maintenance. This approach creates a bidirectional link between documentation and implementation, reducing the likelihood of documentation drift as the codebase evolves.

1.4 Section Headings

Section headings in this document follow a hierarchical numbering scheme to reflect logical organization. The main sections address system architecture, data models, API design, user interface, deployment strategy, and future enhancements. Subsections provide technical depth on specific components or algorithms.

All headings are left-justified and use bold formatting to distinguish them from body text. The numbering system allows unambiguous cross-referencing between sections, which is particularly valuable when discussing interactions between different subsystem components.

1.5 General Guidelines for Preparation of Text

The platform enforces strict data validation at multiple layers. Input sanitization prevents SQL injection and cross-site scripting attacks. Rate limiting protects against denial-of-service attempts. The moderation workflow ensures that inappropriate content never reaches end users, while the audit trail maintains accountability for all administrative actions.

The scoring algorithm deserves special attention due to its central role in the platform's value proposition. The current implementation uses a weighted average that incorporates both the mean rating and the number of reviews, with a Bayesian prior that pulls scores toward a global average when review counts are low. This approach is mathematically sound and produces intuitively reasonable results.

The sentiment analysis module, when enabled, augments reviews with automatically computed sentiment scores ranging from negative to positive. These scores are derived from a transformer model fine-tuned on the LaRoSeDa Romanian sentiment corpus, achieving macro-F1 scores above 0.80 on held-out test data. The sentiment indicators help users quickly assess the emotional tone of reviews without reading full text.

1.6 File Naming and Delivery

The project follows a consistent directory structure that separates concerns across multiple top-level folders: `frontend` for the React application, `backend` for the FastAPI service, `crawler` for data ingestion scripts, and `nlp` for optional sentiment analysis components. Each directory contains its own dependency specifications and configuration files.

Deliverables are organized according to artifact type. Source code resides in version-controlled repositories with clear commit messages and branch naming conventions. Documentation artifacts including this planning report, API specifications, and deployment guides are maintained as markdown or LaTeX files. Binary artifacts such as Docker images are tagged with semantic version numbers and stored in container registries.

1.7 Footnotes

Footnotes are avoided in favor of inline citations or dedicated reference sections. This choice improves readability and ensures that supporting information flows naturally within the main text rather than fragmenting attention across multiple page locations. When additional context is necessary, it is incorporated directly into the paragraph structure.

2 Illustrations

The system architecture can be visualized as a layered diagram with clear separation of concerns. At the bottom layer, PostgreSQL and Meilisearch provide persistent storage and search indexing respectively. The middle layer consists of the FastAPI backend service that orchestrates business logic and exposes REST endpoints. The top layer includes both the React frontend for end users and the crawler subsystem for data ingestion.

All figures in documentation are numbered with Arabic numerals and include descriptive captions. Illustrations are embedded inline near the text that references them, maintaining visual coherence between description and depiction. For web-based documentation, vector formats such as SVG are preferred for scalability across different display resolutions.

The user interface mockups demonstrate the key workflows: discovering books through search, reading reviews and ratings on edition detail pages, submitting new reviews with star ratings and text, and viewing ranking explanations that show how scores are calculated. Each screen includes clear navigation elements and calls-to-action that guide users through their tasks.

3 Equations

The scoring formula used for ranking editions is expressed mathematically to enable precise implementation and verification. Let r_i denote the rating of the i -th review, n denote the total number of reviews for an edition, $\bar{r}$ denote the global average rating across all editions, and m denote the minimum number of reviews required for full confidence. The Bayesian weighted rating R is computed as:

$$R = \frac{n\bar{r}_{\text{edition}} + m\bar{r}}{n + m} \quad (1)$$

where $\bar{r}_{\text{edition}} = \frac{1}{n} \sum_{i=1}^n r_i$ is the arithmetic mean of ratings for the edition. This formula ensures that editions with few reviews are pulled toward the global average, reducing the impact of outliers or manipulation attempts. As the number of reviews increases, the score converges toward the true average rating for that edition.

The confidence metric, displayed alongside rankings, is calculated as:

$$C = \frac{n}{n + m} \quad (2)$$

This value ranges from zero (when $n = 0$) to one (as $n \rightarrow \infty$), providing users with a transparent indication of ranking reliability. High-confidence rankings are based on substantial review volumes, while low-confidence rankings signal that additional reviews could significantly impact the score.

Appendix A: Team Responsibilities

The project workload is distributed evenly across five team members, each owning specific components while contributing to cross-cutting concerns such as testing and documentation. The following allocation ensures clear accountability while enabling collaboration:

Iacob Teodora – Crawler and Ingestion Lead (20% workload): Designs and maintains BeautifulSoup crawlers for multiple Romanian bookstore sources, implements data normalization and deduplication logic, prepares Romanian review text datasets for NLP processing, and supports integration tests for the crawl-to-API pipeline.

Sandu Theodor – Frontend/UI Lead (20% workload): Owns the React interface including all pages and components, implements user workflows for catalog browsing, search, review submission, moderation, and data export, ensures responsive design across desktop and mobile viewports, and integrates sentiment indicators and confidence metrics in ranking displays.

Martinas Ioana Maria – Backend API Lead (20% workload): Owns FastAPI endpoint implementations, request/response validation with Pydantic schemas, security features including authentication and rate limiting, error handling and logging infrastructure, and exposes NLP-derived fields through API responses.

Mercas Ioana Elisabeta – Data Model and Scoring Lead (20% workload): Owns SQLAlchemy model definitions, Alembic migration scripts for schema evolution, score computation logic including Bayesian shrinkage, ranking consistency checks, audit trail correctness, and schema support for sentiment scores and confidence values.

Sacarescu Rebecca Maria – Search, QA, and Release Lead (20% workload): Owns Meilisearch configuration and relevance tuning, end-to-end test scenario development, regression tracking across releases, documentation quality assurance, release readiness verification, and leads NLP evaluation protocol including metrics collection and error analysis.

All members participate in code review, with critical features requiring at least one peer approval before merging to the main branch. Weekly synchronization meetings ensure alignment on priorities and surface integration issues early in the development cycle.

References

- [1] Dumitrescu, S. D., et al. (2021). "LaRoSeDa: A Large Romanian Sentiment Data Set." *Proceedings of the 13th Language Resources and Evaluation Conference*, pp. 2505–2513.
- [2] Tiangolo, S. (2018). "FastAPI: Modern, Fast (High-Performance) Web Framework for Building APIs with Python." Available at <https://fastapi.tiangolo.com>.
- [3] Meilisearch Team. (2021). "Meilisearch: A Lightning Fast Search Engine." Available at <https://www.meilisearch.com>.
- [4] Alembic Contributors. (2023). "Alembic: A Database Migration Tool for SQLAlchemy." Available at <https://alembic.sqlalchemy.org>.
- [5] Wilson, E. B. (1927). "Probable Inference, the Law of Succession, and Statistical Inference." *Journal of the American Statistical Association*, 22(158), pp. 209–212.